

SENAC RJ — Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

SPRINT 3 — FUNDAMENTOS CONCEITUAIS

Tema: DDD — Entendendo o Domínio do Problema

Entrega: 22/05/2026

Este documento apresenta os fundamentos conceituais do Domain-Driven Design (DDD), abordando os conceitos de Domínio, Linguagem Ubíqua e Bounded Context, acompanhados de um Glossário de Domínio aplicado a um sistema fictício de **Gestão de Entregas Logísticas**. O objetivo é construir a base teórica necessária para a modelagem prática desenvolvida nas sprints seguintes.

1. O Que é Domain-Driven Design?

Domain-Driven Design (DDD) é uma abordagem de desenvolvimento de software proposta por Eric Evans no livro homônimo de 2003. Seu princípio central é que a **complexidade do software deve ser enfrentada por meio de um profundo entendimento do domínio de negócio** — o problema que o software resolve — e que esse entendimento deve guiar todas as decisões de modelagem e arquitetura.

O DDD não é um framework nem um conjunto de ferramentas, mas sim um conjunto de padrões, princípios e práticas que promovem a colaboração contínua entre desenvolvedores e especialistas de domínio (domain experts), resultando em um modelo de software que reflete com fidelidade a realidade do negócio.

Pilar	Descrição
Foco no Domínio	O coração do software é o modelo de domínio, não a tecnologia utilizada.
Colaboração Contínua	Desenvolvedores e domain experts trabalham juntos ao longo de todo o projeto.
Iteração do Modelo	O modelo evolui à medida que o entendimento do domínio se aprofunda.
Linguagem Compartilhada	Uma linguagem comum (ubíqua) elimina ambiguidades e alinha equipes.

2. Domínio

No contexto do DDD, o **Domínio** representa a esfera de conhecimento e atividade ao redor da qual a lógica da aplicação é construída. Em outras palavras, é o 'mundo real' que o software pretende modelar e automatizar.

2.1 Subdomínios

Domínios complexos são divididos em subdomínios para facilitar o entendimento e a organização do trabalho. O DDD classifica os subdomínios em três categorias:

Tipo	Definição	Exemplo (Logística)
Core Domain	O diferencial competitivo do negócio. Onde está a vantagem estratégica.	Algoritmo de roteamento de entregas em tempo real.
Supporting Domain	Suporta o core, mas não é o diferencial. Pode ser implementado internamente.	Gestão de cadastro de motoristas e veículos.
Generic Domain	Funcionalidade genérica que pode ser terceirizada ou comprada.	Autenticação de usuários, envio de notificações por e-mail.

2.2 Domínio x Problema x Solução

É importante distinguir o **espaço do problema** (o domínio e seus subdomínios, que existem independentemente do software) do **espaço da solução** (o modelo de domínio e os bounded contexts, que são artefatos do software criados para resolver os problemas do espaço do problema).

3. Linguagem Ubíqua

A **Linguagem Ubíqua** (Ubiquitous Language) é o vocabulário compartilhado entre desenvolvedores e especialistas de domínio dentro de um Bounded Context. É uma linguagem rigorosa e sem ambiguidades que deve ser usada em *todos* os artefatos do projeto: conversas, documentos, código-fonte, testes, diagramas e banco de dados.

A ausência de uma linguagem ubíqua gera um custo de tradução constante entre o vocabulário de negócio e o vocabulário técnico, introduzindo erros e mal-entendidos que se propagam para o código.

3.1 Princípios da Linguagem Ubíqua

Compartilhada	Usada igualmente por devs e domain experts, sem adaptações de lado a lado.
Rigorosa	Cada termo tem um significado preciso e único dentro do contexto.
Viva	Evolui com o entendimento do domínio; novos termos são adicionados conforme necessário.
No Código	Os nomes de classes, métodos e variáveis refletem diretamente a linguagem do negócio.
Documentada	Registrada em um glossário acessível a todos os membros da equipe.

3.2 Exemplo Prático — Sistema de Entregas

Considere a seguinte situação: o departamento de operações usa o termo '*pacote*' para se referir a qualquer item em trânsito, enquanto a equipe de TI usa '*objeto*'. Com a linguagem ubíqua, define-se que o termo correto e único é **Remessa**, que passa a aparecer tanto nas reuniões quanto no código:

```
// Sem linguagem ubíqua — ambíguo
class Item { ... }
class Pacote { ... }
class Objeto { ... }

// Com linguagem ubíqua — claro e alinhado ao domínio
class Remessa { ... }
class EntregaAgendada { ... }
class RoteiroDeEntrega { ... }
```

4. Bounded Context

O **Bounded Context** (Contexto Delimitado) é o padrão mais fundamental do DDD. Ele define uma fronteira explícita dentro da qual um modelo de domínio específico se aplica. Dentro dessa fronteira, todos os termos da linguagem ubíqua têm um significado preciso e inequívoco.

Diferentes bounded contexts podem usar o mesmo termo com significados diferentes. Por exemplo, o termo *Cliente* no contexto de **Vendas** carrega informações de histórico de compras e crédito, enquanto no contexto de **Suporte** o mesmo termo se refere a tickets abertos e nível de satisfação — são modelos distintos para a mesma entidade do mundo real.

4.1 Bounded Contexts do Sistema de Entregas Logísticas

Bounded Context	Responsabilidade	Principais Conceitos
Pedidos	Criação e ciclo de vida dos pedidos de entrega.	Pedido, Item, Endereço de Entrega, Status do Pedido.
Roteirização	Cálculo de rotas otimizadas para as entregas.	Rota, Parada, Janela de Tempo, Veículo, Motorista.
Rastreamento	Monitoramento em tempo real das remessas.	Remessa, Evento de Rastreamento, Localização, ETA.
Notificações	Comunicação com destinatários e remetentes.	Destinatário, Canal de Notificação, Mensagem, Gatilho.
Financeiro	Faturamento e cobrança pelos serviços de entrega.	Fatura, Tarifa, Desconto, Forma de Pagamento.

4.2 Context Map — Relações entre Contextos

O **Context Map** é um diagrama que descreve as relações entre os bounded contexts do sistema. Os principais padrões de integração são:

- **Shared Kernel:** Dois contextos compartilham um subconjunto do modelo. Exige coordenação constante.
- **Customer / Supplier:** Um contexto downstream depende do upstream. O upstream define a API.
- **Conformist:** O downstream aceita o modelo do upstream sem negociação.
- **Anti-Corruption Layer:** O downstream cria uma camada de tradução para se isolar do modelo upstream.
- **Open Host Service:** O upstream expõe um protocolo público bem documentado.

No sistema de entregas, a relação entre **Roteirização** → **Rastreamento** segue o padrão *Customer/Supplier*: o contexto de Rastreamento consome os dados de rota gerados pela Roteirização para atualizar o ETA das remessas.

5. Glossário de Domínio — Sistema de Gestão de Entregas Logísticas

O glossário a seguir define os termos da Linguagem Ubíqua para o sistema fictício de Gestão de Entregas Logísticas. Estes termos devem ser usados uniformemente em código, documentação e comunicação entre a equipe.

Bounded Context: Pedidos	
Termo	Definição
Pedido	Solicitação formal de transporte de um ou mais itens de uma origem a um destino. Possui um ciclo de vida que vai de CRIADO a ENTREGUE ou CANCELADO.
Item	Unidade de carga dentro de um Pedido. Possui peso, dimensões e tipo de manuseio (frágil, refrigerado, etc.).
Endereço de Entrega	Localização física de destino de um Pedido, composta por logradouro, número, complemento, bairro, cidade, estado e CEP.
Remetente	Pessoa física ou jurídica que solicita a entrega. Responsável pelo pagamento do serviço.
Destinatário	Pessoa que receberá o Pedido no Endereço de Entrega.

Bounded Context: Rastreamento	
Termo	Definição
Remessa	Representação física de um Pedido em trânsito. Uma Remessa possui código único de rastreio e gera Eventos de Rastreamento.
Evento de Rastreamento	Registro de uma ocorrência na vida da Remessa: coleta, saída para entrega, entrega realizada, tentativa frustrada.
ETA	Estimated Time of Arrival — estimativa de horário de chegada da Remessa calculada em tempo real.

Bounded Context: Roteirização	
Termo	Definição
Rota	Sequência ordenada de Paradas que um Veículo deve percorrer para realizar um conjunto de entregas.
Parada	Ponto de visita obrigatório em uma Rota, associado a um Pedido e a uma Janela de Tempo.
Janela de Tempo	Intervalo de horário dentro do qual a entrega em uma Parada deve ser realizada.
Motorista	Profissional habilitado responsável por executar uma Rota. Possui CNH, veículo associado e jornada máxima de trabalho.

Veículo	Meio de transporte com capacidade (peso e volume) e tipo (moto, van, caminhão).
----------------	---

Bounded Context: Financeiro	
Termo	Definição
Fatura	Documento de cobrança gerado ao Remetente ao final do ciclo de um Pedido. Contém o valor total calculado pelas Tarifas.
Tarifa	Regra de precificação baseada em parâmetros como distância, peso, prazo e tipo de serviço.

Bounded Context: Notificações	
Termo	Definição
Canal de Notificação	Meio de comunicação usado para contatar o Destinatário: SMS, e-mail ou push notification.
Gatilho de Notificação	Evento do domínio que dispara o envio de uma Notificação: ex. Remessa saiu para entrega.

6. Considerações Finais

Os três conceitos apresentados nesta sprint — Domínio, Linguagem Ubíqua e Bounded Context — formam o alicerce do DDD. Sem um claro entendimento do domínio, os desenvolvedores constroem soluções para o problema errado. Sem uma linguagem ubíqua, surgem mal-entendidos que geram bugs silenciosos. Sem bounded contexts bem definidos, o modelo se torna monolítico e difícil de evoluir.

Na **Sprint 4**, estes conceitos serão colocados em prática por meio da modelagem tática completa do sistema de entregas logísticas, com a identificação de entidades, value objects, agregados e repositórios.